

Derin Öğrenme Tabanlı Öneri Sistemi ile Seyahat Önerilerinin Geliştirilmesi

Enhancing Trip Suggestions with Deep Learning based Recommender System

Necati ERKAL , Nurdan SARAN

Bilgisayar Mühendisliği Bölümü

Çankaya Üniversitesi

Ankara, Türkiye

ne.erkal22@gmail.com, buz@cankaya.edu.tr

Özetçe—Öneri sistemlerinin önemi, verilerin karmaşıklığı nedeniyle son zamanlarda artmaktadır. Kullanıcıların hoşuna gidebilecek öneriler yapmak giderek zorlaşmaktadır. Bu durum, özellikle bir sonraki şehri tavsiye etmenin zorlu bir görev olduğu seyahat öneri sistemlerinde geçerlidir. Derin öğrenmenin çeşitli çalışmalarda tavsiye doğruluğunu artırdığı ve karmaşık verileri ele aldığı gösterilmiştir. Bu çalışma, derin öğrenme tabanlı bir seyahat öneri sistemi için yeni mimariler, veriler ve hiperparametre ayarlama teknikleri sunmaktadır. Çalışma, WSDM WebTour 2021 Yarışması'nda NVIDIA Ekibi'nin kazanan çözümünün algoritmasını ve veri kümesini analiz etmekte ve bu çözüm için geliştirmeler önermektedir.

Anahtar Kelimeler — seyahat tavsiye; WebTour 2021; derin öğrenme tabanlı tavsiye sistemleri. (*key words)

Abstract—The importance of recommender systems has increased recently. It's due to the complexity of the data. It is becoming increasingly difficult to make recommendations that users might like. This is especially true in trip recommender systems, where recommending the next city is a challenging task. Deep learning has been shown to improve recommendation accuracy and handle complex data in various studies. This study presents new architectures, data, and hyperparameter tuning techniques for a deep learning-based trip recommender system. The study analyzes the algorithm and dataset of the NVIDIA Team's winning solution in the WSDM WebTour 2021 Challenge and proposes enhancements to it.

Keywords — trip recommendation; WebTour 2021; deep learning-based recommender systems.

I. GİRİŞ

RecSys olarak da bilinen tavsiye sistemleri çeşitli platformlarda çokça ihtiyaç duyulan uygulamalar olarak hayatımıza girmiştir. NVIDIA, derin öğrenme tavsiye sistemlerine yardımcı olmak için Merlin adlı bir araç geliştirmiştir. NVIDIA'nın ekipleri iki RecSys yarışmasını kazanmıştır: 2020'deki ACM RecSys Challenge ve Booking.com tarafından düzenlenen WSDM WebTour 21 Challenge. Bu çalışma, bir yolcunun önceki rezervasyonlarına dayanarak son varış noktasını tahmin etmeye yönelik bir öneri sistemi sunmaktadır [1].

Tavsiye sistemlerinde derin öğrenme, sinir ağı mimarilerini içerir. Amaçları, öneri algoritmalarının performansını artırmaktır. Bu modeller, girdi verilerindeki özellikleri ve ilişkileri elde etmek için çok katmanlı sinir ağları kullanır. Derin öğrenme, birçok araştırmada tavsiye kalitesinde iyileştirmeler göstermiştir.

Günümüzde her geçen gün artan veri kapasitesi ile bu verileri kullanıcı yararına işlemek ve analiz etmek için akıllı sistemlere duyulan ihtiyaç arttığından derin öğrenme tabanlı tavsiye sistemleri, bu zorluğa işaret etmek ve bilgi toplamayı kolaylaştırmak için çözüm olarak birçok çalışmada önerilmektedir [2]-[3]-[4].

Çalışmamız, kullanıcı davranışlarını ve tercihlerini anlamak için karmaşık modeller ve algoritmik teknikler kullanarak daha iyi kişiselleştirilmiş seyahat önerileri sunmayı amaçlamaktadır. Booking.com tarafından sağlanan karmaşık bir veri kümesi kullanılarak seyahatler ve kullanıcı tercihleri arasındaki ilişkilerin modellenmesine odaklanmaktadır. Sistemi geliştirmek için veri artırma, özellik mühendisliği, mimari ve hiper parametre ayarlama dahil olmak üzere çeşitli deneyler yapılmıştır. Araştırmanın katkıları arasında model optimizasyonu ve hiperparametre ayarlama yer almaktadır. Model optimizasyonu olarak çok katmanlı algılayıcı ve varyasyonları, bir dizi hakkındaki bazı bilgileri hatırlayan basit RNN (simple RNN), GRU gibi çeşitli derin öğrenme mimarilerinin test edilmesi; model performansını en üst düzeye çıkarmak için en uygun parametre kombinasyonlarının bulunması gerçekleştirilmiş ve daha iyi sonuçlar elde edilmiştir. Çalışma sonunda her bir model denemesi analiz edilerek tavsiye sisteminin doğruluğunu ve verimliliğini artıran kritik bulgular elde etmek için sonuçların performans analizi yapılmıştır.

II. OTURUM TABANLI MATRİS ÇARPANLARINA AYIRMA İLE ÇOK KATMANLI ALGILAYICI

Çok Katmanlı Algılayıcı (multilayer perceptron-MLP), tamamen birbirine bağlı nöronların birden fazla katman ile birbirine bağlanması ile oluşan yapay bir sinir ağıdır. Her katman bir veya daha fazla nöron içerir. MLP, ileri beslemeli bir sinir ağıdır; bu, verilerin giriş katmanından gizli katmanlara ve

çıkış katmanına tek yönde gittiği anlamına gelir. Matris çarpanlara ayırma (factorization) ise öneri sistemlerinde kullanılan ana makine öğrenme tekniğidir. Gizli faktörleri keşfetmek için büyük bir kullanıcı ögesi etkileşim matrisini daha düşük boyutlu matrislere böler. Öğrenilen bu faktörler, geçmiş kullanıcı ögesi etkileşimlerine dayalı olarak kişiselleştirilmiş öneriler sağlamak için kullanılır. Oturum Tabanlı Matris Çarpanlara Ayırma (session based matrix factorization-SMF) ise bir kullanıcının oturumunda veya ziyaretinde bir sonraki ögeyi tahmin etmeye odaklanan öneri sistemleri için kullanılan bir modeldir [5] [6] [7].

III. YÖNTEM

Bu çalışmada, WSDM WebTour 2021 Yarışmasında booking.com'un sağladığı [8]' de anlatılan veri setiyle seyahat verileri için tavsiye sistemlerinde birinci olan NVIDIA Ekibinin hibrit algoritmasının ilk algoritması olan MLP-SMF mimarisi incelenmiş, çalışmayı geliştirmek amacıyla bazı çözüm önerileri sunulmuştur.

Çalışmamız iki farklı sistem üzerinde gerçekleştirilmiştir. Ubuntu 22.04 Linux ve Windows 10 işletim sistemine, NVIDIA RTX 3060 GPU' ya ve 32 gigabyte RAM'e sahip bir masaüstü bilgisayar ile başlangıç veri setiyle yapılmış tüm deneyler, NVIDIA RTX 4090 GPU' ya sahip bir sunucu ile veri arttırımı sonrasında elde edilen ve başlangıç veri setinden daha büyük boyutlu diğer veri setlerinin deneyleri icra edilmiştir. Kodu çalıştırmak ve yeni mimariler oluşturmak için UBUNTU tarafında Visual Studio Code (VSCode), Windows tarafında ise veriler üzerinde çalışmak ve arttırılmış veri setleri elde edebilmek için SQL Server Management Studio (SSMS) kullanılmıştır. Veri analizi ve algoritmada yer alan hesaplamalar için RAPIDS açık kaynak yazılım ve donanım hızlandırma platformu bir CONDA ortamı içine kurulmuştur. Deneylerin tamamı Wandb online platformu kullanılarak kayıt altına alınmış ve grafiğe dökülmüştür.

Tahmin başarısını arttırmayı hedefleyen yöntemler üç kısımda ele alınabilir. Birinci kısımda veri setiyle ilgili özellik mühendisliği (feature engineering) ve veri arttırımı (data augmentation), ikinci kısımda model ve mimari, üçüncü kısımda ise hiperparametre ayarlama deneyleri yapılmıştır.

A. Veri Seti

Booking.com'un çevrimiçi platformunda yapılan gerçek seyahat rezervasyonlarının özellikleri; 1.545.502 rezervasyonu ve 39.902 olası destinasyon arasında 359.000 benzersiz seyahati içeren 4-10 arası şehir ziyareti içermektedir. Bu veri seti, insanların seyahat kararlarını nasıl verdiklerini anlamak için yalnızca tamamlanmış gezilerin kullanılması nedeniyle farklıdır. Her veri satırı bir rezervasyonu temsil eder [8].

Sunulan çalışmanın ilk aşaması olan ön işleme (preprocessing) alınarak kullanılmıştır. 'utrip_id' sütunu aracılığıyla bu rezervasyonların hangi seyahate ait olduğu bilgisine erişilebilir. Veri setinde ele alınan problem, gizli olan city_id değerini tahmin ederek son şehri tahmin etmektir.

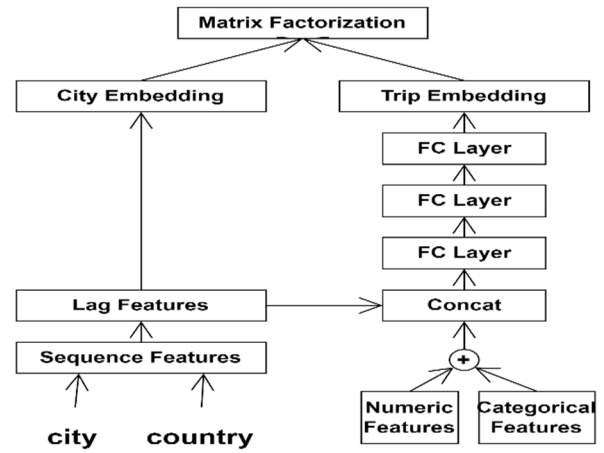
Özellik mühendisliği (feature-engineering) modelin kullanıcı davranışına ilişkin anlayışını geliştirmek için haftanın günü, yılın haftası gibi bilgilerin ve ek ilişkisel özelliklerin çıkarılmasını içeren önemli bir odak noktası olarak var olan

çalışmada ele alınmış, bizim çalışmamızda da 'Next_City', 'Pair_Count' ve 'Possibility' özellikleri oluşturulmuştur.

Veri arttırma (data augmentation) safhasında var olan çalışmaya göre, mevcut veri setindeki seyahatler tersine çevrilmiş ve yeni, tekrar etmeyen seyahatler elde edilmiştir. Bu düşünceden yola çıkarak; veriyi arttırmak amacıyla altı ve üzeri şehir sayısına sahip seyahatlerin ilk ve son ziyaret edilen şehirleri çıkarılmış, yeni seyahatler olarak veri setine eklenmiştir. Üç iterasyonda aynı işlem tekrar edilerek üç farklı arttırılmış veri seti elde edilmiştir. Veri setimiz birinci iterasyon sonucunda 1545502 satırdan 1907466 satıra, ikinci iterasyonda 2096681 satıra ve üçüncü iterasyonda 2172282 satıra yükselmiş ve üç ayrı arttırılmış veri seti deneyi icra edilmiştir. Mevcut çalışmadaki gibi tamamı ters çevrilerek iki katı veriyle çalışılmıştır.

B. Algoritma

Kazanan takımın kullandığı hibrit algoritmanın ilk modeli olan Oturum Tabanlı Matris Faktörizasyonlu Çok Katmanlı Algılayıcı (MLP-SMF) mimarisi Şekil 1'de gösterilmiştir.



Şekil 1: Başlangıç MLP-SMF Mimarisi

Model, seyahat ile ilgili kalınan şehirler ve ülkeleri, otel isimleri gibi kategorik, varış ve ayrılış tarihleri gibi numerik girdiler alır. Buradaki sonraki şehir tavsiye edeceğimiz etiketimizdir. Gömmeler (embeddings), kategorik değişkenlere ve modelin anlayabileceği ayrı öğelere sayısal değerler verilmesine yardımcı olur. Model, bu gömmeleri kullanır ve kategoriler arasındaki ilişkileri anlamak için bunların boyutlarını eşit şekilde ayarlar. Son beş şehri ve ülkelerini tutan gecikme özellikleri (lag features) hem MLP ye input olurken hem de SMF e doğrudan girdi olarak verilir. Bu gecikme özellikleri, modelin son beş şehir ve ülkenin seyahat geçmişini hatırlamasına yardımcı olur. Son beş şehri ve ülkeyi tutmak, ziyaret edilen şehirlerin sırasına göre kalıpların (patterns) öğrenilmesine yardımcı olur. Model tüm bu gömmeleri karıştırır fakat yine de gecikme özelliklerinin yardımıyla şehirlerin sırası korunur. Birleştirilen gömmeler bir dizi katmandan (FC Layer) geçer. Bu adımlar son seyahat oturumumuzun özetini oluşturmaya yardımcı olur (Trip Embedding vektörü). Oluşturulan özetlenmiş oturumla model, farklı şehirlere matris faktörizasyonu yardımıyla puanlar verir. Verilen bu puanlar seyahat geçmişimize dayanarak bir sonraki şehri önermeye

yardımcı olur. En yüksek puana sahip ilk dört şehir öneri olarak sunulur, bir sonraki şehir önerilen dört şehrin arasındaysa öneri doğru olarak kabul edilir.

Mevcut modelde beş katlı çapraz doğrulama (5-fold cross validation) kullanılmıştır. Çapraz doğrulama safhasında veriler beşe bölünmüş, her beş parçadan birisi doğrulama, diğerleri eğitim verisi olarak kullanılmıştır. Bu işlem beş kez her parça doğrulama verisi olarak kullanılabileceği kadar tekrar eder. Puan tüm sonuçların ortalaması alınarak hesaplanır.

[9]'daki çalışmada derin mimarileri optimize etmek için 'ağ kırma' (network trimming) adında bir teknik önerilmektedir. Ağ kırma, aktivasyon göstermeyen ya da etkisi diğer katmanlara göre daha az olan bir katmanın tespit edilmesine yönelik bir çalışmadır. Bu çalışmadan yola çıkarak modeldeki aktivasyon göstermeyen katmanları tespit etmek amacıyla iki ve üçüncü katmanlar sırasıyla çıkarılmış ve etkileri araştırılmıştır. Orta katman etkisi en az katman olarak belirlenmiştir.

[10]'de bir sinir ağına en uygun katman sayısını bulmayı ve modele katman eklemenin genel puan ve model karmaşıklığı üzerine etkilerini araştırarak bir çalışma sunulmuştur. Bu fikirden yola çıkarak, katman eklemenin etkilerini araştırmak amacıyla bir ve iki katman eklenmiş mimariler incelenmiştir.

[11]'de başka bir problemin çözümünde tam bağlı katmanlara sahip GRU modelinin eklendiği bir GRU-FC yaklaşımı sunulmaktadır. Bu yaklaşımın bizim problemimizdeki sonuçlarını gözlemlemek amacıyla etkisi daha az olan orta tam bağlı katman çıkarılmış GRU eklenerek model test edilmiştir. Ortadaki tamamen bağlı katmanın çıkarılması ve tekrarlayan bir sinir ağına yeniden konumlandırılması yoluyla elde edilen mimarilerde RNN modeli de denenmiştir.

C. Hiperparametre Ayarlama

Önerilen çalışmada sunulan parametrelere tek tek değerler verilerek modele etkileri araştırılmış fakat istenilen sonuçlara ulaşılamadığı görülmüştür. Bu sebeple wandb platformu yardımıyla sweep.yaml adında bir dosya oluşturulmuş, Seyreltme (Dropout) 0,05-1, Gömme (Embedding Dimension) 32-128, Dönem (Epoch) 6-24, Gradyan Birikim (Gradient Accumulation) 1-2, Saklı Boyut (Hidden Dimensions) 512-2048, Öğrenme Oranı (Learning Rate) 5,00E-05-0,002, Toptan büyüklüğü (Batch Size) 128-512, İşçi(Workers) 4-16 değer aralıklarının araştırılması bu dosya içerisinde bildirilmiş ve bu dosya kullanılarak hiperparametre deneyleri yapılmıştır.

Bunlarla birlikte kullanılan iyileştiriciler (optimizers); Adam, SGD, AdamW, Nadam, RMSprop, Adagrad, Adadelta'dır. Yitim fonksiyonları (Loss Function); CrossEntropyLoss, MseLoss, TripletMarginLoss, HingeEmbeddingLoss, BCEWithLogitLoss' tur. Terminal vasıtasıyla yaml dosyası çalıştırılmış, parametre ve değer aralıkları Bayes metoduyla seçilerek optimal değer aralıkları belirlenmeye çalışılmıştır.

IV. DENEYSEL SONUÇLAR

Bu bölümdeki veri ve mimari deneyleri için algoritma 5 fold 12 epokta çalıştırılmıştır. Orijinal çalışmada eğitim kaybı değeri 2.902, doğrulama kaybı değeri ise 3.022 olarak gözlemlenmiştir. Üç yinleme (iteration) sonucunda üç versiyonunu elde

ettiğimiz arttırılmış veriler kullanılarak deneyler icra edilmiştir. Deneylerin tamamında başarı beklenenin aksine her veri arttırımında düşmüş, çalışma zamanı beklendiği gibi artmıştır. 1'inci yinleme sonucunda elde edilen veri kullanılarak yapılan deneyde başarı 0.554759'a düşmüş, çalışma zamanı 35 dakika 48 saniyeye yükselmiştir. 2'nci yinlemede başarı biraz daha düşerek 0.548075, çalışma zamanı 36 dakika 11 saniye olarak bulunmuştur. Üçüncü ve son yinlemeyle elde edilen arttırılmış veriyle icra edilen deney sonucunda ise başarı 0.545146, çalışma zamanı 36 dakika 43 saniye olarak bulunmuştur.

Veri arttırımı deneylerinden sonra mimari deneyleri icra edilmiştir. Orijinal modele katman eklenmiş/çıkarılmış, çıkarılan katmanların yerine basit RNN ve GRU modelleri yerleştirilerek modelin bu mimarilerdeki performansları gözlemlenmiştir.

Tablo 1: Model Deney Sonuçları

Name	Skor	Çalışma Zamanı	Eğitim Kaybı	Validasyon Kaybı
Orjinal Model	0.566712	26 m 51 s	2.902	3.022
+1 FC Katman	0.567328	27 m 4 s	2.930	3.009
+2 FC Katman	0.565918	27 m 33s	2.966	3.009
-1 FC 2.Katman	0.562569	37 m 4s	2.906	3.076
-1 FC 3.Katman	0.540499	39m 7s	3.271	3.215
+GRU	0.561364	3 h 6 m 9 s	2.917	3.054
+Basit RNN	0.541391	1 h 22 m 41 s	3.190	3.193

Tablo 1' de tüm mimari çalışmalarının modele etkisi skor, çalışma zamanı, eğitim kaybı (train loss), validasyon kaybı (validation loss) değeri ile kıyaslanmıştır. +1 FC katman ile en iyi sonuçlar alındığından hiperparametre çalışmaları modelin bu hali üzerinde yapılmıştır.

Hiper parametre ayarı ile birçok çalıştırma (deney) yapılmıştır. Tablo 2' de, hiper parametre ayarlama deneyleri arasında en iyi çalışmaların sonuçları gösterilmektedir. 12 epoch, dropout olarak 0.1688, öğrenme oranı olarak 0.00023358, train batch size olarak 452, 8 worker, optimizer olarak Adadelta, hidden dimension olarak 1435, embedding dimension olarak 67, gradient accumulation olarak 2 ve loss function olarak BCEWithLogitsLoss kullanılarak 2.8274 train loss değeri, 3.0409 validation loss değeri ve 43 dakika 37 saniye çalışma ile 0.568032 olan en iyi skor elde edilmiştir.

Tablo 2: Hiperparametre Ayarlama En iyi Skor Veren Deney Sonuçları

Score	Runtime	Dropout	Epochs	LR	train loss	validation loss	train batch size	workers	optimizer	hidden dim	embedding dim	gradient accumulation	loss_fct
0.568032	43m37s	0.1688	12	0.00023538	2.8274	3.0409	452	8	Adadelta	1435	67	2	BCEWithLogitsLoss
0.567976	43m7s	0.2009	13	0.00087881	2.8265	3.0424	163	14	SGD	1055	55	2	TripletMarginLoss
0.567945	41m31s	0.3996	23	0.00110539	2.8268	3.0426	442	15	Nadam	556	111	2	BCEWithLogitsLoss
0.567944	42m58s	0.4351	6	0.00076916	2.8265	3.0413	199	5	rmsprop	1473	97	1	BCEWithLogitsLoss
0.567904	44m8s	0.0729	23	0.00119349	2.8265	3.0369	129	13	adagrad	1892	76	1	TripletMarginLoss
0.567893	44m1s	0.1882	24	0.00081236	2.8270	3.0423	207	15	AdamW	901	104	1	TripletMarginLoss
0.567885	43m	0.8651	23	0.00068224	2.8267	3.0413	334	13	SGD	917	92	1	CrossEntropyLoss
0.567728	42m	0.8711	16	0.00159902	2.8267	3.0384	156	5	RMSProp	1837	128	1	BCEWithLogitsLoss
0.567907	43m42s	0.1377	15	0.00058241	2.8257	3.0408	129	4	Adam	1429	99	2	CrossEntropyLoss
0.567889	44m5s	0.2847	6	0.00027535	2.8274	3.0416	311	13	adagrad	1400	66	1	BCEWithLogitsLoss
0.567834	43m5s	0.9975	9	0.00179221	2.8266	3.0390	159	15	RMSProp	1654	123	2	HingeEmbeddingLoss

V. SONUÇLAR

Çalışmada uygulanan veri arttırımı 4-10 arasındaki şehir gezilerine odaklanılarak gerçekleştirilmiştir. Deneysel sonuçlara göre, veri miktarı arttıkça modelin performansı düşme eğilimindedir. Bu performans düşüşünün, daha karmaşık ve çeşitli veri setlerinin zaten yüksek sayıda olan hedef sayısının artması ve hedeflerin dengesiz dağılmasından (imbalanced) kaynaklandığı düşünülmektedir. Çalışmanın bu kısmı, veri arttırmanın modelin öğrenme dinamiklerini nasıl değiştirebileceğini göstermektedir.

Başlangıç modeli üç katmanlı algılayıcı yapısına sahiptir ve bu yapı deneyler boyunca değiştirilerek sistemin performansı ölçülmüştür. İlk olarak, sırasıyla orta ve üçüncü katmanların kaldırılmasının etkileri değerlendirilmiştir. İkinci katmanın çıkarılması modelin puanında hafif bir düşüşe ve çalışma süresinde artışa neden olurken, üçüncü katmanın çıkarılması puanda daha önemli bir düşüşe ve çalışma süresinde daha da büyük bir artışa neden olmuştur. Bu da katman sayısının azaltılmasının bu modelin genel performansını olumsuz etkilediğini göstermektedir. Daha sonra, ikinci katman yerine GRU ve basit RNN katmanları eklenmesi denemiştir. GRU'nun eklenmesi, modelin puanını orijinaline göre yakın tutarken çalışma süresini önemli ölçüde artırmıştır. Basit RNN eklenmesi de skoru düşürmüş ve çalışma süresini artırmıştır. Bu deneyler, GRU ve basit RNN' nin FC katmanları arasındaki bu özel yapıda bizim problemimizde etkili olmadığını göstermektedir. Doğrusal katmanın eklenmesiyle oluşturulan model, dengeli performans ve kabul edilebilir çalışma süresi ile en iyi sonuçları sunmuştur. Bu model, orijinal model tarafından sağlanan istikrarı korurken biraz daha iyi bir skor sunan en uygun yapı olarak öne çıkmaktadır. Diğer konfigürasyonlar ya performansta düşüşe neden olmuş ya da çalışma süresini önemli ölçüde artırmıştır. Hiperparametre ayarlama deneylerinden elde edilen sonuçlar, performansı etkileyen çeşitli faktörleri de göstermektedir. Birincisi elde edilen puanların yakın aralıkta (0,567834-0,568032) olması, deneylerin genel olarak tutarlı sonuçlar verdiğini göstermektedir. Ancak en yüksek puana 43 dakika 37 saniye gibi bir çalışma zamanında ulaşılmış olması, zaman verimliliği ile performans arasında dikkatli bir denge kurulması gerektiğini göstermektedir. Eğitim ve doğrulama kaybı değerlerinin birbirine yakınlığı, modelin hem eğitim

setinde hem de doğrulama setinde iyi performans elde ettiğini gösterir. Farklı iyileştiriciler arasındaki karşılaştırmada Adadelta, bu deney seti için en iyi sonuçları üretirken, bazı iyileştiricilerin belirli bir veri kümesi ve model mimarisi için diğerlerinden daha etkili olabileceğini göstermektedir. Bu deneyler parametre değerlerinin gerçek kombinasyonunun, modelin genel performansı üzerinde etkiye sahip olabileceğini göstermektedir. Bu çalışmada olduğu gibi bazı öneri sistemleri zamanla değişen bağlamsal bilgiler içerebilir bu tür veri setlerinde kullandığımız veri artırma teknikleri kullanılabilir. Bu geçici bilgiler, zaman içindeki kullanıcı tercihlerindeki eğilimleri analiz edebilir. E-ticarete etkili öneri sistemleri, müşterilere tercihlerine, göz atma geçmişine ve geçmiş satın alma işlemlerine göre ilgili ürünleri önererek satışların artmasına yol açabilir.

KAYNAKLAR

- [1] McDonald C. and Moreira G., "How to build a winning recommendation system, part 1," NVIDIA Technical Blog, 26-Apr-2021. [Online]. Available: <https://developer.nvidia.com/blog/how-to-build-a-winning-recommendation-system-part-1/>. [Accessed: 15-Nov-2023].
- [2] Mu R., "A survey of recommender systems based on deep learning," *IEEE Access*, 6: 69009–69022, 2018.
- [3] Covington P., Adams J., and Sargin E., "Deep neural networks for YouTube recommendations," in Proceedings of the 10th ACM Conference on Recommender Systems, 2016.
- [4] Kiran R., Kumar P., and Bhasker B., "DNNRec: A novel deep learning-based hybrid recommender system", *Expert Syst. Appl.*, vol. 144, no. 113054, p. 113054, 2020.
- [5] Desai M. and Shah M., "An anatomization on breast cancer detection and diagnosis employing multi-layer perceptron neural network (MLP) and Convolutional neural network (CNN)," *Clin. eHealth*, vol. 4, pp. 1–11, 2021.
- [6] Almeida L. B., "Multilayer perceptrons," in Handbook of Neural Computation, IOP Publishing Ltd, 2004.
- [7] Finzi M., Welling M., and Wilson A. G., "A practical method for constructing equivariant multilayer perceptrons for arbitrary matrix groups," *arXiv [cs.LG]*, 2021.
- [8] Goldenberg D. And Levin P., "Booking.com Multi-Destination Trips Dataset," in Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval, 2021.
- [9] Hu H., Peng R., Tai Y.-W., and Tang C.-K., "Network trimming: A data-driven neuron pruning approach towards efficient deep architectures," *arXiv [cs.NE]*, 2016.
- [10] Uzair M. and Jamil N., "Effects of Hidden Layers on the Efficiency of Neural networks," in 2020 IEEE 23rd International Multitopic Conference (INMIC), 2020.
- [11] Zheng T., Wang Q., Shen Y., and Lin X., "Gradient rectified parameter unit of the fully connected layer in convolutional neural networks," *Knowl. Based Syst.*, vol. 248, no. 108797, p. 108797, 2022.